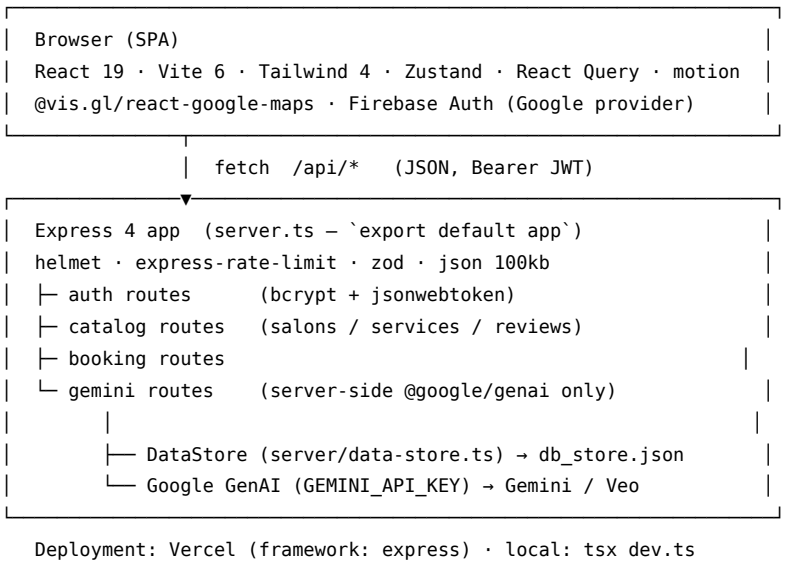
TRD — Leish! Aesthetic Marketplace

Document	02 — Technical Requirements / Design Document
Version	1.0
Date	2026-10-08
Status	Draft for review
Companion	01-prd.md

---

1. Architecture Overview



Key principle: the Gemini API key never reaches the client. Every `/api/gemini/*` call is proxied by Express.

---

2. Stack

Layer	Choice	Version	Notes
UI framework	React	19.0.1	Concurrent features, hooks-
Build	Vite	^6.2.3	`@vitejs/plugin-react`, `@tailwindcss`
Styling	Tailwind CSS	^4.1.14	CSS-first `@theme` config in `src/`
Server	Express	^4.21.2	Single `createApp()` factory, `exp`
Runtime (dev)	tsx	^4.21.0	`dev.ts` boots Vite middleware
Validation	zod	^4.6.5	Shared client/server schema
Server state	@tanstack/react-query	^5.104.1	`staleTime 5m`, `retry 1`
Client state	zustand	^5.0.15	`persist` middleware, partializ
Forms	react-hook-form + @hookform/resolvers	^7.89 / ^5.9	Currently under-used; `lib/fo`
Motion	framer-motion / motion	^14 / ^12	`AnimatePresence`, `layoutId` nav pil
AI	@google/genai	^2.4.0	Server-side only
Maps	@vis.gl/react-google-maps	^1.10.1	`APIProvider`, `Map`, `AdvancedMarker`
Auth storage	firebase	^12.19.0	Google OAuth for Workspac
Security	helmet, express-rate-limit, bcrypt, jsonwebtoken	—	See §7
Language	TypeScript	~5.8.2	`npm run lint` = `tsc --noEmit`

---

3. Application Structure

```
leish-google/
├─ dev.ts                 # Dev/prod bootstrap (Vite middleware vs static)
├─ server.ts              # Express app + all 28 API routes (~1,300 LOC)
├─ server/
│  └─ data-store.ts       # DataStore class, seed data, JSON persistence
├─ db_store.json          # Runtime persistence (salons, bookings, users, reviews)
├─ src/
│  ├─ main.tsx            # React root + QueryClientProvider + ErrorBoundary
│  ├─ App.tsx             # View switch (activeTab machine), Explore view
│  ├─ index.css           # Tailwind @theme — the live design tokens
│  ├─ types.ts            # Shared domain model
│  ├─ components/         # 18 feature components
│  └─ lib/
│     ├─ store.ts         # Zustand (auth, tab, filters, hero image)
│     ├─ api.ts           # fetch wrapper + React Query hooks  ← canonical
│     ├─ firebase.ts      # Firebase init + Google provider
│     ├─ workspace.ts     # 14 Workspace REST helpers
│     ├─ design-tokens.ts # Token documentation (unreferenced)
│     ├─ forms.ts         # Shared RHF+zod hooks (unreferenced)
│     └─ useAuth.ts       # (unreferenced)
├─ index.html             # Fonts, meta, Schema.org JSON-LD
└─ vercel.json            # express framework, build → public/
```

View routing

There is **no URL router**. Navigation is a Zustand-driven view machine:

```ts
type ActiveTab = 'explore' | 'lookbook' | 'ai-stylist' | 'workspace' | 'client' | 'provider'
```

`App.tsx` renders the active tab inside `AnimatePresence`; `SalonDetails` replaces Explore when `selectedSalon != null`. Modals (`BookingModal`, `AuthModal`, `ProfileModal`) are orthogonal overlays.

Technical requirement TR-1: introduce URL-backed routing (`/`, `/listing/:id`, `/lookbook`, `/advisor`, `/workspace`, `/bookings`, `/portal/:section`) so detail pages are shareable and back/forward work. Recommended: `react-router` (or a ~50-line history sync over the existing store).

---

4. Domain Model

Defined in `src/types.ts`, persisted via `server/data-store.ts`.

```
User        { id, name, email, role: client|provider|admin, phone?, avatar?, salonId?, bio?, createdAt }
Salon       { id, type: 'studio'|'mua', name, tagline, description, rating, reviewCount,
              location, address, image, gallery[], category, services[], staff[],
              featured, workingHours{}, artistTitle?, yearsExperience?, kitBrands?,
              travelRadius?, instagramHandle?, startingPrice? }
Service     { id, name, price, duration(min), description, category }
StaffMember { id, name, role, avatar, rating }
Booking     { id, salonId, salonName, salonAddress, serviceId, serviceName, servicePrice,
              serviceDuration, staffId, staffName, date(YYYY-MM-DD), time,
              clientName, clientEmail, clientPhone, notes?, status,
              createdAt, isLocationEvent?, eventVenue?, travelSurcharge?, attachedMoodboard? }
Review      { id, salonId, clientName, rating, text, date }
LookbookItem{ id, lookName, category, vibeDescription, colorPalette[], complexion{},
              eyeArtistry{}, lipFormula{}, longevityFeatures[], groundedTrendContext?,
              recommendedSalonId?, recommendedServiceName?, heroImage?, lightingBestFor? }

BookingStatus = 'pending' | 'confirmed' | 'completed' | 'cancelled'.
```

Persistence

`DataStore` loads `db_store.json` at boot and rewrites the **entire file synchronously** after each mutation.

Environment	Path	Durability
Local dev	`process.cwd()/db_store.json`	Durable
Vercel	`/tmp/db_store.json`	**Ephemeral — wiped every deploy**

Auto-migration runs at boot: if listings carry legacy categories (`hair | nails | massage`) or the salons array is empty, the store re-seeds from `INITIAL_*` constants.

TR-2 (P0): replace the JSON store with a durable, concurrent-safe backend — **Firestore** is the intended target: `firestore.rules` (owner-only `bookings`, public-read `reviews`) and `firebase-blueprint.json` (entity → `/bookings/{id}`, `/reviews/{id}`) already exist, and the Firebase project (`leish-498007`) is configured. Requirements:

- Server-side Firestore Admin SDK (never client-writable for catalog data).
- Idempotent seed/migration job; no destructive re-seed when data exists.
- Rules parity with current API authz; deny-by-default.
- Fallback: managed Postgres (Neon/Supabase) with the same schema if Firestore latency is a concern.

---

5. API Contract

Base: `/api`. JSON in/out. Auth: `Authorization: Bearer <JWT>`.

5.1 Auth

Method	Path	Auth	Notes
POST	`/auth/register`	–	zod `registerSchema`; bcrypt hash (10 rounds); returns user + JWT
POST	`/auth/login`	–	`bcrypt.compareSync`; returns user + JWT
GET	`/auth/me`	JWT	Current user
PUT	`/auth/profile`	JWT	Self-only id check; re-hashes password if changed
GET	`/auth/demo-accounts`	–	⚠ Returns **all users sans password** — must be restricted

5.2 Catalog

Method	Path	Auth	Notes
GET	`/salons`	–	Full listing set
GET	`/salons/:id`	–	Detail
PUT	`/salons/:id`	JWT	⚠ **No ownership check**
POST	`/salons/:id/services`	JWT	⚠ **No ownership check**
DELETE	`/salons/:id/services/:serviceId`	JWT	⚠ **No ownership check**

5.3 Bookings

Method	Path	Auth	Notes
GET	`/bookings?email=&salonId=`	–	⚠ Public read of PII — must require auth
POST	`/bookings`	–	zod `bookingSchema`; ⚠ date regex corrupted (§9.1)
PATCH	`/bookings/:id/status`	JWT	⚠ No provider-ownership verification

5.4 Reviews & venue

Method	Path	Auth	Notes
GET	`/reviews/:salonId`	–	Public read
POST	`/reviews`	–	⚠ Unauthenticated create → review bombing
POST	`/venue/estimate-travel`	–	Deterministic pseudo-distance (string hash), **not** geocoding

5.5 Gemini

Method	Path	Model	Tooling
POST	`/gemini/advice`	`gemini-3.5-flash`	JSON mode + `responseS`
POST	`/gemini/summarize-reviews`	`gemini-3.5-flash`	JSON mode → `{vibe, be`
POST	`/gemini/trends-lookbook`	`gemini-3.5-flash`	`googleSearch` grounding,
POST	`/gemini/quality-audit`	`gemini-3.5-flash`	JSON mode, 7s timeo
POST	`/gemini/chat`	`gemini-3.5-flash` / `gemini-3.1-pro-preview` / `gemini-3.1-flash-lite`	Search or Maps groun
POST	`/gemini/generate-image`	`gemini-3.1-flash-image-preview` → `gemini-3.1-flash-lite-image`	aspect ratio, optional
POST	`/gemini/generate-video`	`veo-3.1-fast-generate-preview` → `veo-3.1-lite-generate-preview`	1 video, 720p, 16:9 |
POST	`/gemini/video-status`	—	`ai.operations.getVideosOpe`
POST	`/gemini/video-download`	—	Streams `video/mp4` with

TR-3: all `/gemini/*` endpoints must (a) validate input with zod, (b) enforce a wall-clock timeout, (c) return `{ source: 'model' | 'fallback', data }` so the UI can label fallback output, and (d) be rate-limited on their own bucket.

---

6. Client Data Flow

```
Zustand (persisted)          React Query                Raw fetch
──────────────────           ───────────                ─────────
currentUser, token    ──►    useSalons/useSalon          gemini/*
activeTab                    useBookings/useReviews      provider mutations ⚠
selectedSalon                useCreateBooking            profile update ⚠
search/category/type         (staleTime 5m, retry 1)
currentHeroImage
```

- `src/lib/api.ts` is the **only** place that injects `Authorization: Bearer <leish_auth_token>` (read from `localStorage`).
- **TR-4 (P0):** eliminate all raw `fetch` calls in components; route every request through `api.ts` (or a typed client generated from the zod schemas). Affected: `ClientPortal.tsx:75`, `ProviderDashboard.tsx:123/142/172/188`, `ProfileModal.tsx:70`.
- **TR-5:** unify hero-image state — the store setter and `localStorage['leish_hero_image']` currently compete.
- **TR-6:** adopt or delete the orphaned modules (`design-tokens.ts`, `forms.ts`, `useAuth.ts`). Recommended: make `forms.ts` the single home for shared zod schemas so client/server validation cannot drift.

---

7. Security Design

Control	Implementation	Status
Password storage	bcrypt, 10 rounds	[OK]
Session	JWT, 7d expiry, Bearer header	⚠ `JWT_SECRET` falls back to `'leis`
Transport headers	Helmet	⚠ CSP disabled when `NODE_EN`
Rate limiting	`/api/auth/*` 20/15min; `/api/*` 100/15min; `trust proxy 1` on Vercel	⚠ single global bucket
Input validation	zod on register/login/booking/review/status/profile	⚠ partial (see §9)
Body limit	`express.json({ limit: '100kb' })`	[OK]
PII stripping	`sanitizeUser()` removes `password`	[OK]
AuthZ (ownership)	—	[NO] missing on salon/servic
Secrets	—	[NO] Firebase web key + Ma

TR-7 (P0) — authorization model:

```
canUpdateSalon(user, salonId)   = user.role === 'provider' && user.salonId === salonId
                                  || user.role === 'admin'
canModifyBooking(user, booking) = booking.clientEmail === user.email        // client cancels
                                  || (user.role === 'provider' && ownsSalon(booking.salonId))
canReview(user, booking)        = completed booking exists for user.email
```

Enforce in middleware (`requireOwnership('salon')`) rather than in each handler.

TR-8: split rate-limit buckets (`auth`, `read`, `ai`) so AI traffic cannot starve login.

TR-9: production requires `JWT_SECRET` with no default; boot fails fast if unset when `NODE_ENV === 'production'`.

---

8. AI Integration Design

- Client: `GoogleGenAI` instantiated **server-side only** from `GEMINI_API_KEY`.
- Every endpoint ships a **deterministic mock fallback** used when the key is missing, equals `MY_GEMINI_API_KEY`, times out, or errors — the UI must remain fully usable offline/keyless.
- Structured output via `responseMimeType: 'application/json'` + `responseSchema` (zod → JSON Schema at the boundary).
- Grounding: `tools: [{ googleSearch: {} }]` for trends, `googleMaps` for location queries.
- Timeout discipline: hard 7s race for user-facing calls; Veo generation is async (operation polling) with client-side poll backoff.

TR-10: normalize Gemini responses through a zod parse before returning; on parse failure, log `raw` and return the fallback rather than a 500.

TR-11: model tiering stays server-side config (`GEMINI_MODEL_*` env), never hardcoded per component.

---

9. Known Defects (must-fix baseline)

These are functional bugs present in the current tree; the Implementation Plan phases them.

> **Status (2026-10-08):** §9.1–§9.4 **FIXED** in commit `412036d` and verified by `server.test.ts` (17 tests) plus a browser E2E of both core journeys. §9.10 and §9.11 were **discovered during that E2E** and are fixed in the working tree (pending commit).

9.1 [HIGH] Booking creation always fails — [OK] FIXED

`server.ts` ~line 102: `date: z.string().regex(/^\d{4}-\d{2}-\d{2}RM/)` — should be `/^\d{4}-\d{2}-\d{2}$/`. The client sends `toISOString().split('T')[0]`, so every request is a 400. `BookingModal` swallows the error (`catch { console.error }`) → **the user sees nothing happen.** *Fix: correct the regex, surface the error in the modal, add an API-level integration test.*

9.2 [HIGH] Markdown-fence stripping corrupted — [OK] FIXED (extracted as `stripJsonFences()`, test-covered)

`server.ts` ~836/838: `.replace(/\s*```RM/, '')` should strip trailing fences (`/```$/`). Fenced JSON from Gemini throws during parse and silently degrades the Lookbook to the hardcoded fallback.

9.3 [HIGH] Malformed ID template literals — [OK] FIXED (verified: `book-1791395804519-8it8ng`)

`server.ts` ~156/401/465: `` `user-${Date.now()}-${Math.random()}.toString(36).slice(2, 8)}` `` — the `.toString(36).slice(2, 8)}` is literal text, producing IDs like `book-1760…-0.84….toString(36).slice(2, 8)}`. *Fix:* *`` `${Date.now()}-${Math.random().toString(36).slice(2, 8)}` ``.*

> The same `}`-for. corruption pattern across 9.1/9.2/9.3 indicates one bad global replace — audit the whole file for the signature.

9.4 [HIGH] Missing `Authorization` on authenticated writes → silent 401 — [OK] FIXED (all routed through `lib/api.ts`; PATCH verified at 200 with `authorization` header)

- `ClientPortal.tsx:75` — cancel booking
- `ProviderDashboard.tsx:123/142/172/188` — status, add/delete service, update salon
- `ProfileModal.tsx:70` — profile update

Only `lib/api.ts` attaches the token. **Provider management, client cancellation and profile editing all fail silently today.**

9.5 [MEDIUM] Orphaned features (built, never mounted)

`ImageStudio.tsx`, `VeoVideoStudio.tsx`, `GeminiChatbot.tsx` are never imported — image, Veo and chat UIs are unreachable despite complete backends. Decision required: wire them in (Phase 2) or remove them.

9.6 [MEDIUM] Decorative claims

- "Firestore Live & Syncing" and "Cloud SQL eligibility verified" badges are static copy.
- Hero stats ("4 Verified Celebrity MUAs", "6 Flagship Beauty Ateliers") are hardcoded strings, not derived from data.
- `/api/venue/estimate-travel` returns hash-derived pseudo-mileage, not geodesic distance.

9.7 [LOW] Locale inconsistency

UI mixes `RM` and `$`; travel returns miles; `index.html` Schema.org declares `USD`. Target: **RM everywhere, km for distance.**

9.8 [LOW] Fonts partially unloaded

`index.html` loads Playfair Display, Plus Jakarta Sans, Space Grotesk, JetBrains Mono; `@theme` declares `Raleway/Lora/Inter` → those fall back to system fonts. Reconcile the two lists.

9.9 [LOW] No tests, no CI, no ESLint

`npm run lint` is `tsc --noEmit` only. `package.json` name is still the template `react-example`. *Partial progress: `npm test` (vitest + supertest) now runs `server.test.ts` — 17 tests covering the booking funnel, auth round-trip, fence stripping and the legacy password migration.*

9.10 [HIGH] Persisted store held PLAINTEXT passwords → every login 401'd — [OK] FIXED

`db_store.json` predates the bcrypt commit: seed writes `hashPassword('password123')`, but the persisted rows contained the literal `password123`. `bcrypt.compareSync(pw, 'password123')` always fails, so **demo sign-in and all logins were broken against the real store** (reproduced as `401 Invalid email or password.`). *Fix: `DataStore.normalizePasswords()` re-hashes any value not matching `/^\$2[aby]\$\d{2}\$/` on load and persists once. Test: "legacy plaintext password migration".*

9.11 [HIGH] Booking date off-by-one (timezone) — booked the wrong day — [OK] FIXED

`getAvailableDates()` built the value with `toISOString().split('T')[0]` (UTC) while the label used `toLocaleDateString` (local). For any local time before the UTC offset — e.g. 00:00–08:00 in GMT+8 — the label read **WED 21 Oct** but the stored date was **2026-10-20**. Verified in the browser E2E: user picks Wednesday, receipt shows Tuesday. *Fix: format `YYYY-MM-DD` from local `getFullYear()/getMonth()/getDate()` components.*

---

10. Performance

Concern	Current	Requirement
Bundle	single `index-*.js` ≈ 1.45 MB	Code-split per route (React.lazy) → initial < 350 KB gzi
Images	remote stock URLs, unoptimized	Lazy-load below fold; fixed aspect boxes to avoid CLS
AI latency	7s timeout race on trends	Show skeleton + fallback label; never block navigation
Data fetching	React Query, staleTime 5m	Keep; add `placeholderData` for smooth tab switches
Re-renders	Explore re-renders on every keystroke	Debounce search input (150–250ms)

---

11. Testing Strategy

Layer	Tool	Scope
Unit	vitest	zod schemas, DataStore transforms, pricing/travel math
API integration	vitest + supertest	Every endpoint: happy path, 400, 401, 403 (ownership!)
Component	vitest + @testing-library	BookingModal 3-step flow, ProviderDashboard status transitions
E2E	Playwright	Guest book flow, provider confirm flow, lookbook → booking
Static	tsc + eslint	`tsc --noEmit` already exists; add ESLint + prettier

Minimum gate before launch: API integration tests covering §9.1–§9.4 and the authz matrix in TR-7.

---

12. Deployment & Environments

Local dev	`npm run dev` → `tsx dev.ts` → Vite middleware + API on `:3000`
Build	`npm run build` → `vite build` → `dist/`
Prod start	`NODE_ENV=production tsx dev.ts` (static `dist/` + SPA fallback)
Vercel	`vercel.json`: `framework: express`, `buildCommand: npm run build && cp -R dist/. public/`, `outputDirectory: public`; ser
Linked project	`leish-google` (org `team_09zaVvx6W9qqXeaJ1Zx6cfGS`)

Required env (server): `GEMINI_API_KEY`, `JWT_SECRET`, `FIREBASE_*` (post-migration), optional `GEMINI_MODEL_*`. **Required env (client):** `VITE_GOOGLE_MAPS_API_KEY` (remove the hardcoded fallback), `VITE_FIREBASE_*`.

TR-12: CI pipeline — `tsc --noEmit` → lint → unit/integration tests → `vite build` → preview smoke test. Block deploys on failure.

---

13. Migration & Evolution Path

1. **Now → stable:** fix §9.1–§9.4, add authz middleware, keep JSON store for local dev only.
2. **Data:** Firestore (rules + blueprint already in repo) behind a `DataStore` interface so the JSON implementation remains the test double.
3. **Routing:** URL-backed routes (TR-1) with redirects from the current tab machine.
4. **Surfaces:** decide on the three orphan studios; mount or delete.
5. **Scale:** if query complexity grows (search/filters/analytics), move catalog to Postgres while keeping Firestore for bookings/reviews if latency allows.
6. **Commerce (v2):** payments, deposits, cancellation policies → requires a ledger and provider onboarding/KYC.

---

14. Summary of Technical Requirements

ID	Requirement	Priority
TR-1	URL-backed routing	P1
TR-2	Durable datastore (Firestore) behind an interface	P0
TR-3	Gemini endpoint contract: zod-in, timeout, `{source, data}`, own rate bucket	P0
TR-4	Single HTTP client — no raw fetches in components	P0
TR-5	Single hero-image state source	P2
TR-6	Adopt or delete orphan modules; share zod schemas client/server	P1
TR-7	Ownership-based authorization middleware	P0
TR-8	Split rate-limit buckets	P1
TR-9	Fail-fast `JWT_SECRET` in production	P0
TR-10	zod-parse all model output; fallback on parse failure	P1
TR-11	Model tiering via env config	P2
TR-12	CI with typecheck + tests + build gate	P1